Algorithmique

Contents

I. Les structures de base d'un langage de programmation 2
A. La déclaration des variables 2
1. Les types de Variables 2
B. La séquence d'instructions 4
1. L'affectation 4
2. Les opérations 5
3. Le dialogue avec l'utilisateur 5
C. La structure alternative 6
D. La Structure répétitive 7
1. La boucle TantQue 7
2. La Boucle Répéter Jusqu'à 7
3. La Boucle Pour 8

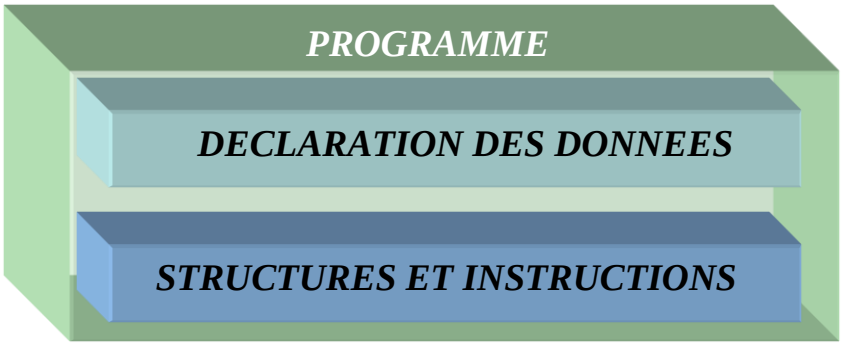
I. *Les structures de base d'un langage de programmation*

Dans ce qui suit nous allons utiliser un pseudo-language, comportant toutes les structures de base d'un langage de programmation. Un programme est une suite d'instruction exécutées par la machine.

Ces instructions peuvent soit s'enchaîner les unes après les autres, on parle alors de ***SEQUENCE D 'INSTRUCTIONS*** ; ou bien s'exécuter dans certains cas et pas dans d'autres, on parle alors de ***STRUCTURE ALTERNATIVE*** ; ou se répéter plusieurs fois, on parle alors de ***STRUCTURE REPETITIVE***.

A. La déclaration des variables

Un programme est composé de deux parties :



- La partie instruction contient les instructions à exécuter.
- La partie données contient toutes les variables utilisées par le programme.

Un programme exécutable est chargé dans la mémoire centrale de l'ordinateur, les valeurs que l'on a affectées aux variables doivent être conservées tout le temps du déroulement du programme. Par conséquent il faut que le programme soit capable de réserver la place nécessaire aux variables.
Pour ce faire, les variables doivent être déclarées afin que le compilateur sache quelle place elles vont occuper.

1. <u>Les types de Variables</u>

Les variables que l'on utilise dans le programme ne sont pas toutes de même nature, il y a des nombres, des caractères, … on dit que les variables sont typées.

Il est nécessaire de donner un type aux variables, car cela permet d'une part de contrôler leur utilisation (on ne peut pas deviser un caractère par un nombre) et d'autre part cela indique quelle place il faut réserver pour la variable.

Généralement les variable peuvent être des types suivants :

- ***Entier***

Il s'agit des variables destinées à contenir un nombre entier positif ou négatif. Dans notre Pseudo-langage, on écrira la déclaration des variables de type entier comme suit :

Variable1, Variable2… : **Entier**

Généralement un entier occupe deux octets, ce qui limite les valeurs de –32768 à +32768. Cependant cela dépend des machines, des compilateurs et des langages. Certains langages distinguent les entiers courts (1 octets), les entiers longs (4 octets) et les entiers simples (2 octets).

Variable1, Variable2… : **Entier Simple**
Variable1, Variable2… : **Entier Long**

- ***REEL***

Il s'agit des variables numériques qui ne sont pat des entiers, c'est à dire qui comporte des décimales. Généralement un nombre réel est codé sur 4 octets. Dans notre pseudo-langage, la déclaration des variables de ce type sera comme suit :

Variable1, Variable2,… : **REEL**

- ***CARACTERE***

Les variables de type caractère contient des caractères alphabétiques ou numériques (de 0 à 9), mais dans ce cas ils ne sont pas considérés comme étant des nombres et on ne peut pas faire d'opérations dessus. Un caractère occupe 1 octet.

Dans notre pseudo-langage, une variable de type caractère de déclare ainsi :

Variable1, Variable2,… : **CAR**

- BOOLEEN

Il est souvent nécessaire lorsque l'on écrit un programme d'introduire des variables qui prennent les valeurs Vrai ou Faux ou les valeurs Oui ou Non.
Pour cela Il Existe un type particulier dont les variables ne peuvent prendre que 2 valeurs : **Vrai** ou **Faux.**
Dans notre pseudo-langage Ce type de variable se déclare de la manière suivante.

Variable1, Variable2,… : **BOOLEEN**

B. La séquence d'instructions

Une instruction est une action que l'ordinateur est capable d'exécuter. Chaque langage de programmation fournit une liste des instructions qui sont implémentées et que l'on peut donc utiliser sans les réécrire.

Dans notre pseudo–langage, nous n'aurons que la liste minimum d'instructions, nécessaire et suffisante pour les programmes que nous aurons à écrire.

1. <u>**L'affectation**</u>

Variable ← Valeur

Se qui se lit « Variable reçoit valeur » et qui signifie que l'on mémorise la valeur à un endroit nommé variable.

<u>Par exemple **:**</u>

I ← 5 ; Termine ← Vrai

2. Les opérations

Les opérations arithmétiques courantes sont l'addition, soustraction, multiplication et division. Ces opération s'écrivent de la manière suivante :

Variable ← Val1 + Val2
Variable ← Val1 - Val2
Variable ← Val1 * Val2
Variable ← Val1 / Val2

On doit toujours affecter le résultat d'une opération dans une variable

Par exemple :

A ← 5 + 2 ; B ← A – 1 ; C ← (B / 2) – 1 + A

3. Le dialogue avec l'utilisateur

Pour permettre au programme de dialoguer avec l'utilisateur, c'est à dire d'afficher un résultat à l'écran et de lire une entrée au clavier, il faut au moins deux instructions une pour lire et l'autre pour afficher. Dans le pseudo-language, elles s'écrivent ainsi :

Lire Variable
Lire Variable1, Variable2,….

Afficher ''Texte''
Afficher Variable
Afficher ''Texte'' , Variable

La première lit tous les caractères qui sont saisis au clavier, jusqu'à ce que l'utilisateur appuie sur la touche entrée, et stocke le résultat dans la variable. La seconde affiche sur l'écran le ou les textes et la valeur des variables.

Par exemple :

Afficher ''Quel est ton nom ? : ''
Lire Nom
Afficher ''Ton nom est :'' , Nom
Afficher ''Le mien est Khalil''

C. La structure alternative

Il est souvent nécessaire lorsque l'on écrit un programme de distinguer entre plusieurs cas conditionnant l'exécution de telles ou telles instructions. Pour ce faire, on utilise une structure alternative (Conditionnelle) : Si on est dans tel cas on fait cela sinon on fait ceci. La syntaxe de cette structure est la suivante :

SI Condition **ALORS** Actions1 [**SINON** Actions2] **FINSI**
NB : Les crochets signifient que la partie sinon est facultative.

❖ ***Les conditions***

Pour exprimer les conditions on utilise les opérateurs conditionnels suivants :
= égal ; **<** Inférieur ; **>** Supérieur ; **<=** Inférieur ou égal ; **>=** Supérieur ou égal ;
<> différents ;

Par exemple :

(A**<**1) (A**<=**B) ……..

On peut combiner des conditions à l'aide des opérateurs logiques suivant : **Ou** ; **Et** ; **XOR**;

Par exemple :

(A<>B) **ET** (A>=5)
((A<0) **ET** ((B=0) **OU** (C<>A)) **XOR** (D=1))

Lorsque l'on écrit de telles conditions, il est recommandé de mettre toutes les parenthèses afin d'éviter les erreurs.

Par exemple :

Si (A<0) **Alors** ABS ← A* (-1) **Sinon** ABS ← A **FINSI**
Si (A<>0) **Alors** ABS ← B/A **FINSI**

❖ ***Les actions***

Les actions qui suivent le Sinon ou le Alors peuvent être :

- Une simple instruction
- Une suite d'instructions

- Une autre structure alternative
- Une autre structure répétitive

D. La Structure répétitive

Un programme à presque toujours pour rôle de répéter la même action un certain nombre de fois. Pour ce faire on utilise une structure permettant de dire « Exécuter telles actions jusqu'à ce que telle condition soit remplie » .
Bien qu'une seule soit nécessaire, la plupart des langages de programmation proposent trois types de structures répétitive. Voici celle de notre pseudo-langage.

1. **La boucle TantQue**

```
TantQue  Condition  Faire
    Actions
Fin TantQue
```

Ce qui signifie tant que la condition est vraie, on exécute les actions.

Par exemple :

```
Fini ← Faux
TantQue  (Non Fini ) Faire
        I← I+1
        SI (X/I < epsilon) Alors
                Fini ← Vrai
        Sinon
                X ← X/I
        FinSi
FinTQ
```

2. **La Boucle Répéter Jusqu'à**

```
Répéter
    Actions
Jusqu'à  Condition
```

Ce qui signifie que l'on exécute les actions jusqu'à ce que la conditions soit vraie.

📽 Par exemple :

```
Fini ← Faux
Répéter
        I ← I+1
        Si (X/I < epsilon ) Alors
        Fini ← Vrai
        Sinon
        X ← X/I
        FSI
Jusqu'à (Fini = Vrai)
```

3. La Boucle Pour

Très souvent, on utilise une structure répétitive avec un compteur et on s'arrête lorsque le compteur a atteint sa valeur finale.

📽 Par exemple :

```
I=1
TantQue (I<10) Faire
        P=P*P
        I=I+1
FinTQ
```

C'est pourquoi la plupart des langages de programmation offrent une structure permettant d'écrire cette répétitive plus simplement. Dans le pseudo-langage c'est la structure POUR :

POUR Variable **Allant de** Valeur initiale **A** Valeur finale [**PAS** Valeur du pas] **Faire**
    Actions
Fin Pour

Lorsque le pas est Omis, il est supposé égale à (+1).

📽 Par exemple :

```
Pour I Allant de 1 A 10 Faire
    P=P*X
Fin Pour
```